

Documentacao Tecnica

Memorias - Anuario Digital

API Go + Chi + PostgreSQL + sqlc + JWT

Item	Descricao
Projeto	Anuario digital para perfis de alunos, galeria e eventos da turma.
Entrega	Sprint 3 / Projeto final de Desenvolvimento de Sistemas Web II.
Autores	Vinicios David Martins Bezerra e Weuler dos Santos Barbosa.
Versao	1.0 - 2026

Este documento resume o MVP implementado, a arquitetura, o modelo de dados, a API, as decisoes de seguranca, a estrategia de testes e o roteiro de demonstracao.

1. Visao geral e escopo

O Memorias - Anuario Digital e uma aplicacao web para registrar e visualizar perfis de alunos, fotos vinculadas e eventos de uma turma. O sistema combina uma interface publica para consulta visual do anuario com uma API administrativa protegida por JWT para operacoes de escrita e leitura sensivel.

O MVP atende aos requisitos principais da entrega: login com JWT, rotas protegidas por middleware, refresh token com rotacao, persistencia em PostgreSQL usando consultas tipadas no pacote db, relacao 1:N entre alunos e fotos, testes automatizados e pipeline de CI.

Funcionalidades entregues

- Visualizacao da pagina inicial com alunos, galeria, eventos e painel administrativo.
- Login administrativo em /api/v1/auth/login retornando access token e refresh token.
- CRUD de alunos e eventos com validacao de entrada e tratamento padronizado de erros.
- Cadastro de fotos associadas a alunos, demonstrando relacao 1:N.
- Rotas de alunos protegidas por Bearer JWT e autorizacao por papel admin.
- Refresh token opaco, armazenado por hash SHA-256 e rotacionado a cada uso.

Papeis de usuario

Papel	Acesso
Visitante	Visualiza o site, fotos estaticas, eventos publicos e os cards dos alunos renderizados no frontend.
Admin	Apos login, usa JWT para acessar rotas protegidas e executar CRUD de alunos, fotos e eventos.

2. Arquitetura da solução

A arquitetura segue separação em camadas. O frontend é servido como arquivos estáticos em public/. A API HTTP é implementada em Go usando chi, middlewares de segurança e controllers que delegam persistência para uma interface Store. Em desenvolvimento local, a Store pode usar memória; em ambiente com DATABASE_URL, usa PostgreSQL.

Camada	Responsabilidade	Arquivos principais
Frontend	HTML, CSS e JavaScript para interface do anuário e painel admin.	public/index.html, public/app.js, public/styles.css
Rotas	Registro de endpoints, middlewares globais, proteção JWT e arquivos estáticos.	cmd/api/main.go
Controllers	Validação HTTP, parse de JSON, respostas e códigos de erro.	interno/controller/*.go
Autenticação	JWT HS256, senha, refresh token opaco e rotação.	interno/auth/*.go
Middlewares	Headers de segurança, rate limit, logging, JWT e autorização.	interno/middleware/*.go
Repositório	Contrato Store e implementações em memória/PostgreSQL.	interno/repository/*.go
Banco	Schema SQL, queries sqlc e tipos gerados.	interno/db/*.go, interno/db/*.sql

Fluxo principal

1. O usuário acessa a página inicial. 2. O admin autentica em /auth/login. 3. O frontend armazena tokens no localStorage. 4. Requisições protegidas incluem Authorization: Bearer . 5. O middleware valida assinatura, expiração e papel. 6. O controller chama a Store. 7. A Store executa consultas em memória ou PostgreSQL. 8. A resposta é serializada em JSON.

Decisão importante: as rotas /api/v1/alunos e /api/v1/public/alunos retornam 401 sem JWT. A visualização de visitante dos alunos é renderizada pelo frontend sem expor a listagem por endpoint público.

3. Modelo de dados e persistencia

O banco relacional usa PostgreSQL e e migrado automaticamente no inicio da aplicacao quando DATABASE_URL esta configurado. O schema cobre administradores, alunos, eventos, fotos e refresh tokens. A relacao 1:N exigida ocorre entre alunos e fotos: um aluno pode possuir varias fotos, e cada foto pertence a um aluno.

Tabela	Campos principais	Observacoes
admins	id, usuario, senha_hash, role, criado_em	Conta administrativa. Senha armazenada como hash bcrypt.
alunos	id, nome, foto, turma, criado_em	Entidade principal do anuario.
fotos	id, aluno_id, url, legenda, enviado_em	aluno_id referencia alunos(id) com ON DELETE CASCADE.
eventos	id, titulo, descricao, data, imagem_url	Linha do tempo publica, com CRUD admin.
refresh_tokens	id, admin_id, token_hash, expires_at, revoked_at	Armazena apenas hash do refresh token e permite rotacao.

Uso do sqlc

As consultas ficam em interno/db/query.sql e sao utilizadas por interno/repository/postgres.go por meio do pacote db. O objetivo e manter SQL explicito com tipos Go previsiveis, reduzindo erros de mapeamento em runtime.

Exemplo de dados aninhados

GET /api/v1/alunos/{id} retorna os dados do aluno e um array fotos. Esse formato demonstra a agregacao da relacao 1:N no controller/repository antes da resposta JSON.

```
{"id":22,"nome":"Aluno Teste API","foto":"/uploads/alunos/ana.jpg","turma":"2026.1","fotos":[{"id":3,"aluno_id":22,"url":"/uploads/alunos/ana.jpg","legenda":"Foto de perfil"}]}
```

4. API REST e autenticao

A API e exposta em /api/v1. Endpoints de login e refresh possuem rate limiting. Eventos podem ser lidos publicamente; operacoes administrativas exigem token Bearer com papel admin. A leitura de alunos pela API tambem e protegida, atendendo ao requisito de rotas protegidas.

Metodo	Rota	Auth	Descricao
POST	/api/v1/auth/login	Publica	Recebe usuario e senha; retorna access token e refresh token.
POST	/api/v1/auth/refresh	Refresh	Rotaciona refresh token e gera novo par de tokens.
GET	/api/v1/alunos	Bearer admin	Lista alunos cadastrados.
POST	/api/v1/alunos	Bearer admin	Cria aluno com nome, foto e turma.
GET	/api/v1/alunos/{id}	Bearer admin	Retorna aluno com fotos aninhadas.
PUT	/api/v1/alunos/{id}	Bearer admin	Atualiza aluno.
DELETE	/api/v1/alunos/{id}	Bearer admin	Remove aluno e fotos vinculadas.
POST	/api/v1/alunos/{id}/fotos	Bearer admin	Cria foto vinculada ao aluno.
GET	/api/v1/eventos	Publica	Lista eventos ordenados por data.
POST/PUT/DELETE	/api/v1/eventos	Bearer admin	CRUD administrativo de eventos.

Login e refresh

O access token e um JWT HS256 com subject, usuario, role, iat e exp. O refresh token e opaco, gerado aleatoriamente, salvo apenas como hash SHA-256 no banco, e revogado na rotacao. Reutilizar um refresh token antigo deve resultar em 401 Unauthorized.

Tratamento de erros

Entradas invalidas retornam 400 Bad Request. Recursos inexistentes retornam 404 Not Found. Ausencia ou invalidade de JWT retorna 401 Unauthorized. Falta de papel adequado retorna 403 Forbidden.

5. Seguranca implementada

A entrega inclui correcoes alinhadas a riscos comuns citados no OWASP. A protecao nao depende de uma unica camada: combina validacao de entrada, autenticacao, autorizacao, rate limiting, headers de seguranca e controle de ciclo de vida de tokens.

Controle	Implementacao	Risco mitigado
JWT Bearer	Middleware AutenticacaoJWT valida formato, assinatura, expiracao e claims.	Acesso nao autenticado a rotas protegidas.
Autorizacao por papel	AutorizarRole("admin") valida role nas claims.	Acesso administrativo indevido.
Rate limiting	NewRateLimiter aplicado em login e refresh.	Tentativas repetidas de login e abuso de autenticacao.
Headers de seguranca	nosniff, DENY frame, no-referrer e CSP.	Clickjacking, sniffing e reducao de superficie XSS.
Validacao/sanitizacao	Inputs de aluno, evento e foto sao validados antes da persistencia.	Dados invalidos, URLs perigosas e payloads inesperados.
Refresh token rotacionado	Token antigo e revogado apos uso e armazenado apenas por hash.	Reuso de token vazado.

Decisao de exposicao dos alunos

A pagina de visitante mostra os cards de alunos por dados estaticos do frontend. As rotas de API /api/v1/alunos e /api/v1/public/alunos continuam protegidas por JWT e retornam 401 sem Authorization. Essa separacao permite uma vitrine publica sem abrir endpoint de listagem sensivel.

Boas praticas operacionais

- Trocar JWT_SECRET em producao e usar valor com pelo menos 16 caracteres.
- Definir ADMIN_USER e ADMIN_PASSWORD no ambiente de deploy.
- Usar HTTPS no deploy para proteger tokens em transito.
- Configurar DATABASE_URL para persistencia real em PostgreSQL.

6. Testes automatizados e CI

A suite automatizada cobre autenticação, controllers, middlewares, repositório em memória e integração PostgreSQL quando TEST_DATABASE_URL está disponível. O CI do GitHub Actions sobe PostgreSQL 16 como serviço e executa `go test ./... -v -count=1`.

Area	O que valida
Auth	Hash de senha, geração/validação de JWT, assinatura alterada e hash de refresh token.
Controller	Listagem, criação, atualização, remoção, erros 400/404, login e refresh com rotação.
Middleware	Headers de segurança e rate limiter.
Router	Rotas de alunos retornam 401 sem JWT e 200 com JWT admin.
Repository	CRUD em memória e teste de integração PostgreSQL quando configurado.

Comandos usados

```
go test ./... -v -count=1
```

Cenário manual recomendado

- Fazer login com admin/admin123 e guardar access_token e refresh_token.
- Chamar GET /api/v1/alunos sem token e confirmar 401.
- Criar aluno com POST /api/v1/alunos usando Bearer token e confirmar 201.
- Criar foto em POST /api/v1/alunos/{id}/fotos e buscar GET /api/v1/alunos/{id}.
- Forçar validação com aluno sem nome e confirmar 400.
- Buscar ID inexistente e confirmar 404.
- Usar refresh token, receber novo par e tentar reutilizar o antigo para confirmar 401.

7. Execucao, configuracao e entrega

A aplicacao pode rodar em modo memoria para desenvolvimento local ou em PostgreSQL para persistencia real. Quando DATABASE_URL esta ausente, a Store em memoria inicializa dados de demonstracao. Quando DATABASE_URL esta presente, a aplicacao migra o schema e cria admin padrao se necessario.

Variavel	Finalidade	Exemplo
DATABASE_URL	Conexao PostgreSQL. Sem ela, usa memoria.	postgres://anuario:anuario@localhost:5432/anuario?sslmode=disable
JWT_SECRET	Segredo HMAC para JWT.	uma-chave-local-segura
ADMIN_USER	Usuario admin inicial.	admin
ADMIN_PASSWORD	Senha admin inicial.	admin123
PORT	Porta HTTP.	8080

Passos locais

```
go mod download
go test ./... -v -count=1
go run ./cmd/api
```

Credenciais padrao

Usuario: admin
Senha: admin123

Observacao: em producao, as credenciais devem vir de variaveis de ambiente. O JWT_SECRET tambem deve ser alterado para um segredo forte e privado.

8. Deciso es tecnicas, limitacoes e evolucao

Principais decisoes

- Go e chi foram escolhidos para manter uma API simples, performatica e com middlewares explicitos.
- sqlc foi usado para preservar SQL claro e ter tipos Go derivados das consultas.
- A interface Store permite alternar entre memoria e PostgreSQL sem mudar controllers.
- JWT curto e refresh token rotacionado equilibram usabilidade e seguranca.
- O frontend permanece sem framework para reduzir complexidade de build e facilitar demonstracao.

Limitacoes atuais

- A visualizacao de visitante dos alunos e alimentada por dados estaticos do frontend para manter a API protegida.
- Nao ha upload binario real; fotos sao cadastradas por URL/caminho local permitido.
- Nao ha papeis alem de admin nesta versao do MVP.
- A integracao PostgreSQL local depende de TEST_DATABASE_URL para executar teste completo fora do CI.

Proximos passos

- Criar endpoint publico seguro para perfis anonimizados, se o requisito de API publica voltar a existir.
- Adicionar storage real para upload de imagens.
- Implementar usuarios por aluno e permissoes granulares.
- Adicionar documentacao OpenAPI e colecao Postman/Insomnia.
- Melhorar observabilidade com logs estruturados e metricas.

Conclusao

O MVP entregue atende ao nucleo tecnico esperado para a sprint final: autenticacao, autorizacao, rotas protegidas, relacao 1:N, persistencia, seguranca basica, testes e CI. A arquitetura em camadas deixa o projeto simples de demonstrar e viavel para evolucao incremental.